torch.autograd.gradcheck.gradgradchec

https://pytorch.org/docs/stable/generated/torch.autograd.gradcheck.gradgradc

Description:

Check gradients of gradients computed via small finite differences against analytical gradients wrt tensors in inputs and grad_outputs that are of floating point or complex type and with requires_grad=True. This function checks that backpropagating through the gradients computed to the given grad_outputs are correct. The check between numerical and analytical gradients uses allclose(). The default values are designed for input and grad_outputs of double precision. This check will likely fail if they are of less precision, e.g., FloatTensor.

Function Signature

```
torch.autograd.gradcheck.gradgradcheck(func, inputs,
grad_outputs=None, *, eps=1e-06, atol=1e-05, rtol=0.001,
gen_non_contig_grad_outputs=False, raise_exception=True,
nondet_tol=0.0, check_undefined_grad=True,
check_grad_dtypes=False, check_batched_grad=False,
check_fwd_over_rev=False, check_rev_over_rev=True,
fast_mode=False, masked=False)[source]#

Parameters

Returns

Return type
```

Returns:

bool

Notes & Warnings

NOTE Note The default values are designed for input and grad_outputs of double precision. This check will likely fail if they are of less precision, e.g., FloatTensor.

⚠ WARNING Warning If any checked tensor in input and grad_outputs has overlapping memory, i.e., different indices pointing to the same memory address (e.g., from torch.Tensor.expand()), this check will likely fail because the numerical gradients computed by point perturbation at such indices will change values at all other indices that share the same memory address.

Detailed Documentation

Documentation

Check gradients of gradients computed via small finite differences against analytical gradients wrt tensors in inputs and grad_outputs that are of floating point or complex type and with requires_grad=True.

This function checks that backpropagating through the gradients computed to the given grad_outputs are correct.

The check between numerical and analytical gradients uses allclose().

The default values are designed for input and grad_outputs of double precision. This check will likely fail if they are of less precision, e.g., FloatTensor.

If any checked tensor in input and grad_outputs has overlapping memory, i.e., different indices pointing to the same memory address (e.g., from torch.Tensor.expand()), this check will likely fail because the numerical gradients computed by point perturbation at such indices will change values at all other indices that share the same memory address.

func (function) – a Python function that takes Tensor inputs and returns a Tensor or a tuple of Tensors

inputs (tuple of Tensor or Tensor) – inputs to the function

grad_outputs (tuple of Tensor or Tensor, optional) – The gradients with respect to the function's outputs.

eps (float, optional) – perturbation for finite differences

atol (float, optional) – absolute tolerance

rtol (float, optional) – relative tolerance

gen_non_contig_grad_outputs (bool, optional) – if grad_outputs is None and gen_non_contig_grad_outputs is True, the randomly generated gradient outputs are made to be noncontiguous

raise_exception (bool, optional) – indicating whether to raise an exception if the check fails. The exception gives more information about the exact nature of the failure. This is helpful when debugging gradchecks.

nondet_tol (float, optional) – tolerance for non-determinism. When running identical

inputs through the differentiation, the results must either match exactly (default, 0.0) or be within this tolerance. Note that a small amount of nondeterminism in the gradient will lead to larger inaccuracies in the second derivative.

check_undefined_grad (bool, optional) – if True, check if undefined output grads are supported and treated as zeros

check_batched_grad (bool, optional) – if True, check if we can compute batched gradients using prototype vmap support. Defaults to False.

fast_mode (bool, optional) – if True, run a faster implementation of gradgradcheck that no longer computes the entire jacobian.

masked (bool, optional) – if True, the gradients of unspecified elements of sparse tensors are ignored (default, False).

True if all differences satisfy allclose condition